Übungsblatt 03: POP3 und HTTP mit Python Sockets

PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

Raphael Bleier, Gruppe YY, WS 2026/2027

Aufgabe 3.1

POP3 anhand RFC 1939 untersuchen: Arbeitsablauf, Sicherheit und Unterschied zwischen single-line und multi-line response. Mit IMAP vergleichen.

Nach dem TCP-Verbindungsaufbau schickt der Server eine Begrüßung mit `+OK`. Es folgen drei Zustände:

1. AUTHORIZATION: `USER` und `PASS` übermitteln die Anmeldung. Der Server prüft die Daten und sperrt den Maildrop exklusiv. Erst bei Erfolg beginnt TRANSACTION (RFC 1939, Abschnitt 4).
2. TRANSACTION: `STAT` nennt Anzahl und Gesamtgröße; `LIST` listet Größen und `RETR n` liefert Nachricht n. `DELE n` markiert eine Nachricht zunächst nur. `RSET` entfernt diese Markierungen (Abschnitt 5).
3. UPDATE: `QUIT` aus TRANSACTION übernimmt die Löschungen und beendet die Verbindung. Bei einem Abbruch ohne dieses `QUIT` darf der Server die Markierungen nicht übernehmen (Abschnitt 6). `QUIT` vor der Anmeldung beendet die Verbindung ohne UPDATE.

Unverschlüsseltes POP3 (üblicherweise auf Port 110) schützt weder Passwort noch Nachrichten. Ein Mitschnitt kann beides offenlegen. Für echte Zugänge ist TLS mit Zertifikatsprüfung nötig, etwa implizites TLS auf Port 995. APOP aus RFC 1939 ist keine Verschlüsselung der Maildaten und kein Ersatz für TLS. Der hier implementierte Server ist auf einen lokalen Labortest ohne TLS beschränkt.

Eine single-line response enthält genau eine Statuszeile mit `+OK` oder `-ERR`. Eine erfolgreiche multi-line response beginnt ebenfalls mit einer Statuszeile, enthält anschließend Datenzeilen und endet mit einer Zeile, die nur einen Punkt enthält. Jede Zeile endet mit CRLF. Datenzeilen mit führendem Punkt erhalten einen zusätzlichen Punkt; der Client entfernt beim Lesen genau diesen einen Punkt. `LIST` ist ohne Argument mehrzeilig, `LIST 1` einzeilig. Eine negative Antwort hat keinen anschließenden Datenblock.

IMAP verwendet getaggte Kommandos und Abschlüsse sowie ungetaggte Antworten. Literale können eine explizite Bytelänge haben; sie werden nicht durch POP3-Dot-Stuffing eingerahmt. IMAP unterstützt serverseitige Ordner, Flags und Synchronisierung mehrerer Geräte. POP3 konzentriert sich auf den Abruf. Serverkopien können dabei erhalten bleiben, wenn der Client keine Löschung verlangt. Grundlage: RFC 9051 sowie LV-Folien 36–37.

Aufgabe 3.2

Einen POP3-Client mit Standardbibliothek und Socket-Kommunikation schreiben. Anmelden, alle Nachrichten auflisten, eine Nachricht abrufen und die Verbindung ordentlich abbauen. Mit einem geeigneten Server testen; lokale Server oder ein TLS-Tunnel sind erlaubt.

Abgabe: `pop3_client.py`. Der Ablauf lautet Begrüßung, `USER`, `PASS`, `LIST`, `RETR n`, `QUIT`. Statusfehler werden abgefangen, CRLF und Dot-Stuffing ausgewertet. Das Passwort wird verdeckt abgefragt. Der Client schließt Reader und Socket auch nach einer fehlerhaften Begrüßung oder einem fehlgeschlagenen `QUIT`.

Der Test gegen den lokalen Server mit der Original-Datenbank bestätigte vier Nachrichten. `LIST` meldete 184, 185, 154 und 164 Bytes. Der erste `RETR`-Abruf stimmte bytegenau mit der CRLF-normalisierten Originalnachricht überein. Weitere Tests prüfen fragmentierte Antworten, Punktzeilen, EOF und die Abwehr von Zeilenumbrüchen in Kommandos. Eine Gegenprobe mit dem im Blatt verlinkten GoPop3Server bestätigte Anmeldung, `LIST`, `RETR` und `QUIT`. Dieser separate Referenzserver lieferte zwei Nachrichten mit 151 und 115 Bytes; der erste Betreff war „Hello Alice“. Der Versuch lief in einem isolierten lokalen Netzwerk.

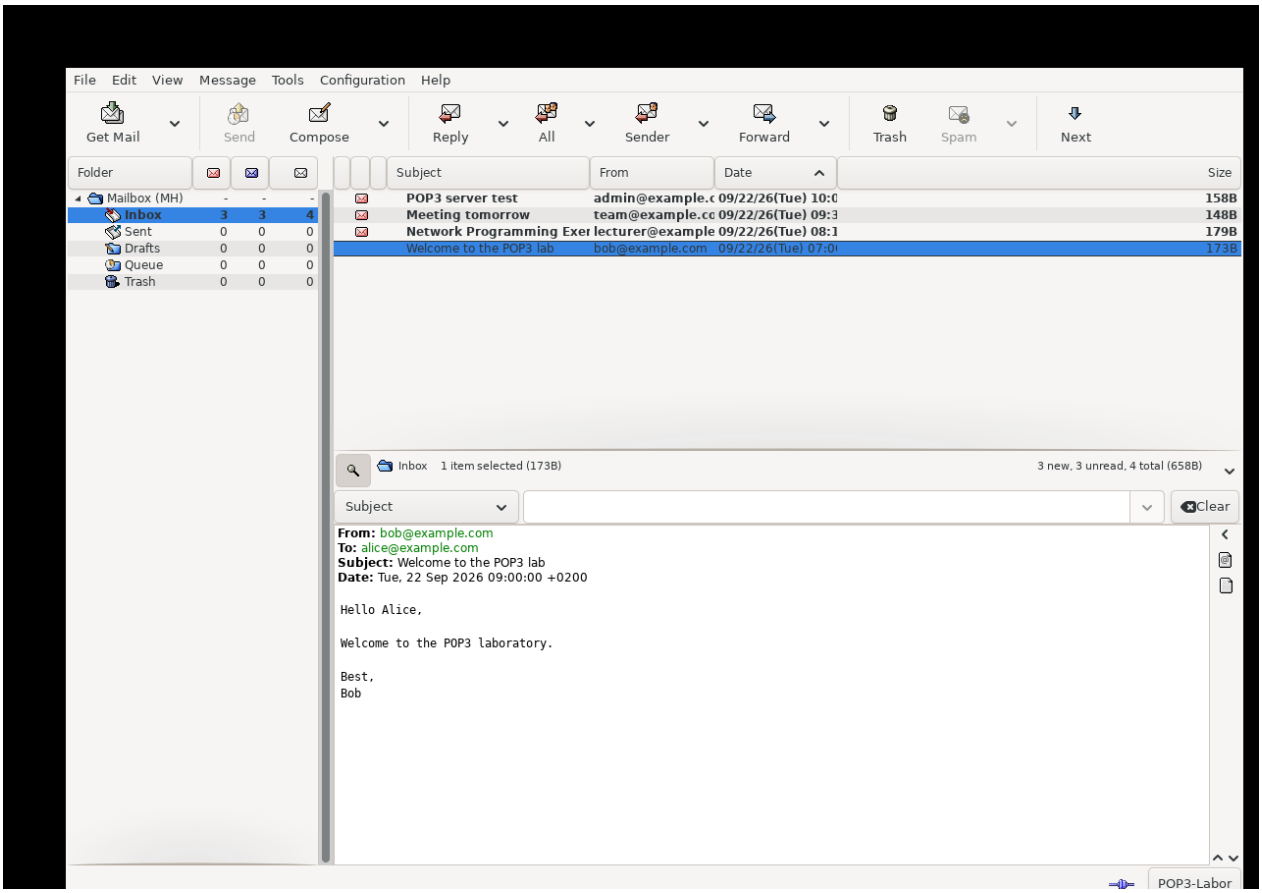
Aufgabe 3.3

Einen minimalen POP3-Server mit der bereitgestellten Klasse SampleDataBase.py implementieren. Die Nachrichten sollen im Posteingang eines echten Mailprogramms erscheinen. Pro Benutzer einen Thread verwenden; parallele Benutzer zulassen. Client und Server praktisch testen und den Ablauf mit Screenshots dokumentieren. RFC 1939 und RFC 2449 beachten.

Abgabe: `pop3_server.py`. Die externe Datenbank wird über `authenticate` und `get_messages` benutzt. Pro Verbindung läuft ein Thread. Ein Lock pro Benutzer verhindert zwei gleichzeitige Transaktionen auf demselben Maildrop. Unterschiedliche Benutzer bleiben parallel bedienbar.

Implementiert sind `USER`, `PASS`, `STAT`, `LIST`, `RETR`, `DELE`, `RSET`, `NOOP` und `QUIT`; zusätzlich `CAPA`, `TOP` und `UIDL`. Nachrichtennummern bleiben innerhalb einer Sitzung stabil. `UIDL` nutzt einen Hash aus Original-ID und Inhalt. Löschungen werden nur bei `QUIT` übernommen. Die Datenbankdatei bleibt unverändert; ohne dauerhafte Löschschnittstelle besteht der Löschstand nur im laufenden Serverprozess. Das Inaktivitätstimeout beträgt 600 Sekunden.

GUI-Test: Claws Mail 4.3.1 mit einem frischen Profil, POP3 auf Loopback, Port 11110, Testbenutzer aus der Original-Datenbank und ohne TLS. Die Option zum Löschen nach Abruf war ausgeschaltet. Nach dem Abruf erschienen vier Nachrichten im Posteingang. Die erste Nachricht wurde geöffnet. Der Screenshot zeigt die Betreffe sowie Header und Text dieser Nachricht. Die Standardbibliothek `poplib` wurde zusätzlich als unabhängiger Protokollclient verwendet.



Aufgabe 3.4

Einen Single-Thread-HTTP/0.9-Server schreiben. Port (Vorgabe 80) und document root als Kommandozeilenparameter anbieten. Einzelne Dateien liefern, für Verzeichnisse index.html verwenden und bei Fehlern ohne Rückmeldung schließen. Mit documentRoot.zip und unterschiedlichen Browsern testen. Werden die Daten richtig angezeigt?

Abgabe: `http_server.py -mode 09`. Dieser Modus behandelt Verbindungen nacheinander. Akzeptiert wird `GET /pfad` mit CRLF. Die Antwort besteht nur aus den Dateibytes; danach wird die Verbindung geschlossen. Pfade werden dekodiert, aufgelöst und auf das Basisverzeichnis beschränkt. Fehlende Dateien und ungültige Anfragen liefern keine Antwortbytes.

Socket-Tests bestätigen `index.html`, Unterverzeichnisse und binäre Dateiinhalte. Kodierte Traversal-Versuche und Symlinks außerhalb des document root werden abgewiesen. Der document root muss während des Betriebs unter eigener Kontrolle bleiben; der Server bietet keine Betriebssystem-Sandbox gegen gleichzeitige fremde Dateisystemänderungen.

Mit Chromium 156.0.8078.4 und Firefox 157.0 wird die Seite in diesem Modus nicht richtig angezeigt. Die Browser senden aktuelle Request-Zeilen, die der strikte Zweitoken-Parser ablehnt. Chromium meldete `ERR_EMPTY_RESPONSE`; Firefox lieferte im Navigationsaufruf `NS_ERROR_NET_RESET`. Der Firefox-Screenshot zeigt die leere Antwort als Browserfehler. Dieses Testergebnis wurde nicht als erfolgreiche Darstellung gewertet.

Aufgabe 3.5

Den Server erweitern: jede Anfrage durch einen neuen Thread behandeln, HTTP/1.0 und persistente Verbindungen unterstützen. Zeitpunkt, Request-Typ, Client-IP und Clientport zentral in einem

eigenen Thread alle fünf Sekunden ausgeben. Geeignet synchronisieren.

Abgabe: `http_server.py -mode 10`. Ein Verbindungsthread liest vollständige Anfragen. Für jede Anfrage startet er einen eigenen Worker. Vor dem nächsten Request auf derselben Verbindung wartet er auf dessen Antwort. Andere Verbindungen arbeiten parallel. `GET` liefert Dateidaten, `HEAD` nur die passenden Header.

Antworten verwenden `HTTP/1.0`, `Content-Length`, `Content-Type` und `Connection`. Nur explizites Keep-Alive hält die Verbindung offen; `close` hat Vorrang. Die Grenze liegt bei 100 Requests beziehungsweise fünf Sekunden Inaktivität. HTTP/1.1-Request-Zeilen werden für moderne Clients akzeptiert, aber mit HTTP/1.0 beantwortet. Request-Bodies, Chunked-Encoding und doppelte Header werden abgelehnt. Fehler erhalten 400, 403, 404 oder 405 entsprechend der Ursache.

Die Handler schreiben ausschließlich Logereignisse in eine threadsichere Queue. Ein separater Logger-Thread sammelt sie und schreibt alle fünf Sekunden Zeitstempel, Methode, Clientadresse und Clientport. Beim regulären Beenden wird der Rest ausgegeben. Nur dieser Thread schreibt den Logstrom. Passwörter und Header werden nicht erfasst.

Im HTTP/1.0-Modus lieferten Chromium und Firefox den Text „It works...!“ und beide Bilder korrekt. Die Bildprüfung bestätigte geladene Bilder mit positiver natürlicher Breite. Zwei Requests auf derselben TCP-Verbindung und 16 parallele Requests wurden separat getestet. Die Browser-Aufnahmen liegen im Ordner `screenshots`; das Erklär-PDF enthält zusätzlich die Chromium-Darstellung.

Prüfung, Quellen und Abgabe

21 Tests liefen mit Python 3.14.7 erfolgreich durch; Browsersteuerung mit Python 3.11.15. Getestet wurden Anmeldung, Dot-Stuffing, Größen, Sperren, Lösch-Commit, Rollback, Framing, Pfadgrenzen, HTTP-Fehler, Keep-Alive, Parallelität und Logging. Startbefehle und Einzelheiten stehen in `README.md`. Die Programme und Tests sind Teil der Abgabe.

Grundlagen: Moodle-Blatt 03; bereitgestellte Datenbankschnittstelle und document root; LV-Unterlage `rn-chapter-2`, Folien 10–11 und 36–37. Normative Quellen: RFC 1939, Abschnitte 3–7; RFC 2449, Abschnitte 4–6; RFC 1945, Abschnitte 5–6; RFC 8314 zur TLS-Einordnung und RFC 9051 zu IMAP.

Moodle nennt für beide Gruppen den 17.11.2026, 08:00 Uhr (Stand 19.10.2026). Die Gruppennummer ist noch nicht bekannt und bleibt als YY markiert. Es wurde nichts auf Moodle eingereicht.

KI-Kennzeichnung laut Aufnahmefolien: Hermes Agent v0.21.5, Modell `gpt-6.1-sol`, verwendet am 19.10.2026 für Code, Texte und Tests. Die Python-Dateien nennen die jeweiligen Arbeitsaufträge. Alle genannten Testresultate und Screenshots stammen aus tatsächlich ausgeführten Programmen.